

Clustering

CARLOS RIVERO
ROCHESTER INSTITUTE OF TECHNOLOGY
DEPT. OF COMPUTER SCIENCE

R·I·T

These slides correspond to the unit of [clustering](#).

Definition



Cluster analysis or clustering is the task of grouping a set of objects such that objects in the same group (cluster) are more similar (in some sense) to each other than to those in other groups (clusters). - Wikipedia

2

Definition of the problem.

Roadmap

1. **Introduction**
2. The K-Means algorithm
3. Evaluating clusters
4. Conclusions

3

This is the roadmap we will follow.

What is a cluster?

“(…) The notion of cluster cannot be precisely defined. Clustering is in the eye of the beholder, and as such, researchers have proposed many induction principles and models whose corresponding optimization problem can only be approximately solved by an even larger number of algorithms.”

- Vladimir Estivill-Castro: Why so many clustering algorithms: a position paper. SIGKDD Explorations 4(1): 65-75 (2002).

4

There is no definition of what a cluster is.

Cluster models

- Connectivity models based of connectivity distance
- Centroid models based on central individuals and the distance to other individuals
- Density models based on connected and dense regions in a space
- Graph-based models based on cliques and their relaxations

5

Some examples of cluster models are those above.

Roadmap

1. Introduction
- 2. The K-Means algorithm**
3. Evaluating clusters
4. Conclusions

A good source of info

CHECK THIS OUT!



<http://stanford.edu/~cpiech/cs221/handouts/kmeans.html>

7

Besides these slides, a very good source of information is Piech's handout.

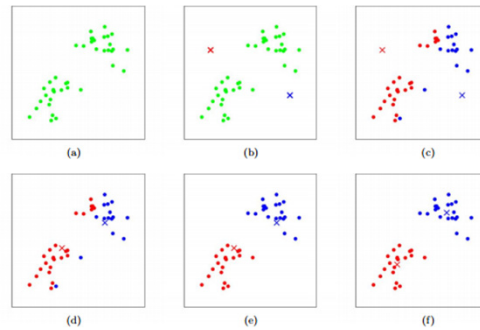
Basic idea

- We are going to work with points in a given space $\mathbb{R}^n$
- Given a k (the number of clusters to be generated), we store k centroids that will define the central points of the cluster
- We assign points to clusters
- We reassess centroids based on current assignment

8

The basic idea...

Basic idea in action



9

Figure (a) represents the original points and (c) the clustering after random assignment and how we progress.

Formal statement

- A set of points $x^{(1)}, \dots, x^{(m)}$
- Each $x^{(i)} \in \mathbb{R}^n$
- Our goal: compute $\mu_1, \dots, \mu_k$ centroids and a label $c^{(i)}$ for each point
- Each $\mu_i \in \mathbb{R}^n$
- $\|x^{(i)} - \mu_i\|$ is the Euclidean distance between the points, i.e., $\text{Sqrt}((x^{(i)}[0] - \mu_i[0])^2 + (x^{(i)}[1] - \mu_i[1])^2 + \dots)$
- Any distance can be used

10

Definitions needed.

Algorithm

1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ randomly.
2. Repeat until convergence: {
 For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

 For each j , set

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}.$$

}

11

The k-means algorithm.

Pseudo-code in Python-like (I)

```
# Function: K Means
# -----
# K-Means is an algorithm that takes in a dataset and a constant
# k and returns k centroids (which define clusters of data in the
# dataset which are similar to one another).
def kmeans(dataSet, k):

    # Initialize centroids randomly
    numFeatures = dataSet.getNumFeatures()
    centroids = getRandomCentroids(numFeatures, k)

    # Initialize book keeping vars.
    iterations = 0
    oldCentroids = None

    # Run the main k-means algorithm
    while not shouldStop(oldCentroids, centroids, iterations):
        # Save old centroids for convergence test. Book keeping.
        oldCentroids = centroids
        iterations += 1

        # Assign labels to each datapoint based on centroids
        labels = getLabels(dataSet, centroids)

        # Assign centroids based on datapoint labels
        centroids = getCentroids(dataSet, labels, k)

    # We can get the labels too by calling getLabels(dataSet, centroids)
    return centroids
```

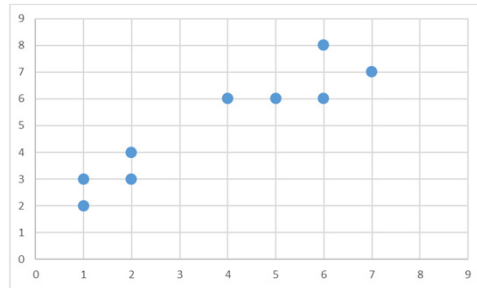
Pseudo-code in Python-like (II)

```
# Function: Should Stop
# -----
# Returns True or False if k-means is done. K-means terminates either
# because it has run a maximum number of iterations OR the centroids
# stop changing.
def shouldStop(oldCentroids, centroids, iterations):
    if iterations > MAX_ITERATIONS: return True
    return oldCentroids == centroids

# Function: Get Labels
# -----
# Returns a label for each piece of data in the dataset.
def getLabels(dataSet, centroids):
    # For each element in the dataset, chose the closest centroid.
    # Make that centroid the element's label.

# Function: Get Centroids
# -----
# Returns k random centroids, each of dimension n.
def getCentroids(dataSet, labels, k):
    # Each centroid is the geometric mean of the points that
    # have that centroid's label. Important: If a centroid is empty (no points hav
e
    # that centroid's label) you should randomly re-initialize it.
```

Example (I)



Example (II)

- Points: $(1, 2), (1, 3), (2, 3), (2, 4), (4, 6), (5, 6), (6, 6), (6, 8), (7, 7)$
- Manhattan distance: $d((a, b), (x, y)) = |a - x| + |b - y|$
- $k = 2$
- We are going to limit to $x^{(i)} \in \mathbb{N}^2$
- Random centroids: $\mu_1 = (2, 8), \mu_2 = (8, 1)$

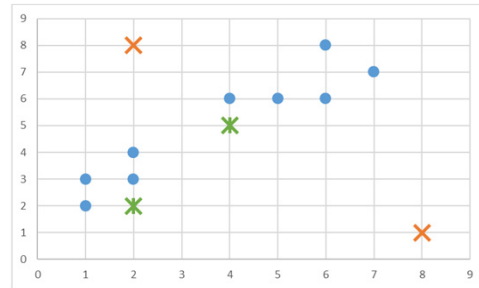
Example (III)

- Distances (μ_1 | μ_2):

	1	2	4	5	6	7
2	<u>2</u> 8					
3	<u>6</u> 9	<u>5</u> 8				
4		<u>4</u> 9				
6			<u>4</u> 11	<u>5</u> 8	<u>6</u> 7	
7						<u>6</u> 7
8					<u>4</u> 9	

- Recomputing centroids: $\mu_1 = (34/9, 45/9) = (4, 5)$, $\mu_2 = (2, 2)$
(randomly reinitialized since no points were assigned to μ_2)

Example (IV)



Example (V)

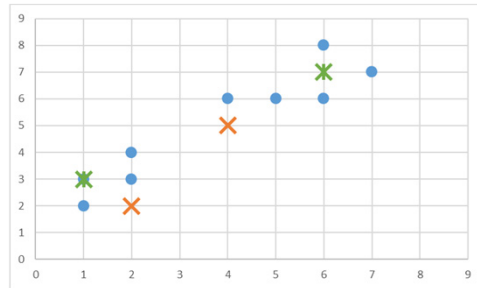
- Centroids: $\mu_1 = (4, 5)$, $\mu_2 = (2, 2)$

- Distances (μ_1 | μ_2):

	1	2	4	5	6	7
2	6 1					
3	5 2	4 1				
4		3 2				
6			1 6	2 5	3 8	
7						5 10
8					5 10	

- Recomputing centroids: $\mu_1 = (28/5, 33/5) = (6, 7)$, $\mu_2 = (6/4, 12/4) = (1, 3)$

Example (VI)



Example (VII)

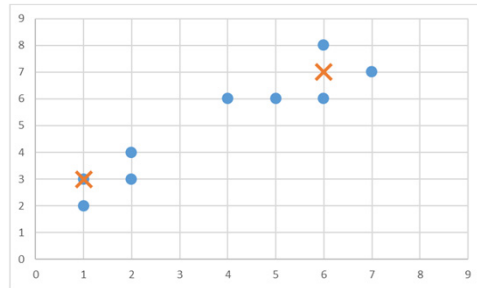
- Centroids: $\mu_1 = (6, 7)$, $\mu_2 = (1, 3)$

- Distances (μ_1 | μ_2):

	1	2	4	5	6	7
2	10 <u>1</u>					
3	8 <u>0</u>	8 <u>1</u>				
4		8 <u>2</u>				
6			<u>3</u> 6	<u>2</u> 7	<u>1</u> 8	
7						<u>1</u> 10
8					<u>1</u> 10	

- Recomputing centroids: $\mu_1 = (28/5, 33/5) = (6, 7)$, $\mu_2 = (6/4, 12/4) = (1, 3)$; same centroids so stop

Example (VIII)



Roadmap

1. Introduction
2. The K-Means algorithm
- 3. Evaluating clusters**
4. Conclusions

Sum of squared errors

$$SSE = \sum_{i=1}^k \sum_{x_j \in C_i} (x_j - \mu_i)^2$$

23

This is used as a method of measuring the variation within clusters.

Example

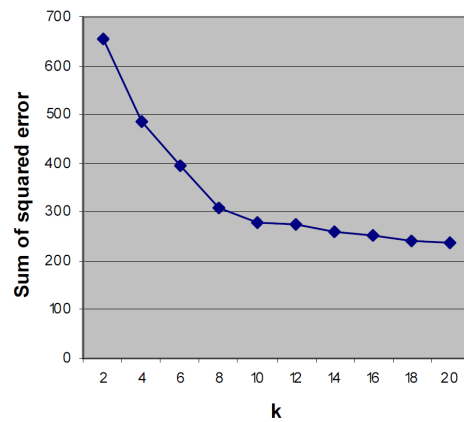
- Centroids: $\mu_1 = (6, 7)$, $\mu_2 = (1, 3)$. $\{(1, 2), (1, 3), (2, 3), (2, 4)\}$ are assigned to μ_2 , and $\{(4, 6), (5, 6), (6, 6), (6, 8), (7, 7)\}$ are assigned to μ_1

	1	2	4	5	6	7
2	1					
3	0	1				
4		2				
6			3	2	1	
7						1
8					1	

- $SSE = 1^2 + 0^2 + 1^2 + 2^2 + 3^2 + 2^2 + 1^2 + 1^2 + 1^2 = 22$

24

Finding a good k

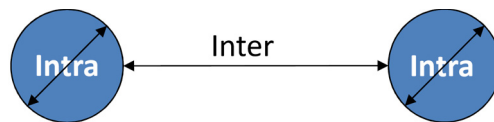


25

Note that in k-means we need to provide a k (the number of clusters); however, it is unclear how many clusters we should have for a given set of points. We need to find what it is called the “knee”: the region where the change in slope begins to level off.

Intra- and inter-cluster distances

- There are two criteria to assess cluster quality:
 - Intercluster dissimilarity
 - Intracluster similarity
- Ideally, the distance between clusters should be large, while the distance within clusters should be very small



Silhouette coefficient

- For each $x^{(i)}$, $a(x^{(i)})$ is the average distance between $x^{(i)}$ and all the points classified in cluster $c^{(i)}$ (same cluster as $x^{(i)}$)
- For each $x^{(i)}$ and cluster $c^{(k)}$ (other than $c^{(i)}$), let $d(x^{(i)}, c^{(k)})$ the average distance between $x^{(i)}$ and all the points classified in $c^{(k)}$
- Let $b(x^{(i)}) = d(x^{(i)}, c^{(k)})$ such that it is minimum
- $S(x^{(i)}) = b(x^{(i)}) - a(x^{(i)}) / \max\{a(x^{(i)}), b(x^{(i)})\}$
- $S \in [-1, 1] = \sum S(x^{(i)}) / m$ (recall that we have m points)
- The closer $S = 1$ the better

Example

- C1: (1, 2), (1, 3); C2: (3, 4), (4, 5); C3: (7, 7), (8, 7)
- $S((1, 2)) = 5 - 1 / 5$; $S((1, 3)) = 4 - 1 / 4$
- $S((3, 4)) = 3.5 - 2 / 3.5$; $S((4, 5)) = 5.5 - 2 / 5.5$
- $S((7, 7)) = 6 - 1 / 6$; $S((8, 7)) = 7 - 1 / 7$
- $S = (4/5 + 3/4 + 1.5/3.5 + 3.5/5.5 + 5/6 + 6/7)/6$
 $= (0.8 + 0.75 + 0.43 + 0.64 + 0.83 + 0.86)/6 = 0.72$

Mutual information (I)

- Let $X = \{x^{(1)}, \dots, x^{(m)}\}$ be a set of elements (points in our case)
- Let $U = \{U_1, \dots, U_R\}$ and $V = \{V_1, \dots, V_C\}$ be two partitions, where U_i ($i=1..R$) and V_k ($k=1..C$) are clusters of elements
- These partitions are also called assignments, e.g., $x^{(1)} \in U_1$, $x^{(2)} \in U_1$, $x^{(3)} \in U_2 \dots$

29

Adapted from: https://en.wikipedia.org/wiki/Adjusted_mutual_information

Mutual information (II)

- Each partition is pairwise disjoint, i.e., a given elements cannot belong to two or more clusters at the same time
- More formally, $U_i \cap U_k = \emptyset$ and $V_i \cap V_k = \emptyset$
- Each partition is complete, i.e., all elements are found in U and V
- More formally, $U_1 \cup U_2 \dots \cup U_R = X$ and $V_1 \cup V_2 \dots \cup V_C = X$

Mutual information (III)

- The probability of an element assigned to a cluster U_i (similarly V_k) is:
 - $P_U(i) = |U_i|/m$ (similarly $P_V(k) = |V_k|/m$)
- The entropy associated with a partition is:
 - $H(U) = -\sum_{i=1}^R P_U(i) \log P_U(i)$ (similarly $H(V) = -\sum_{k=1}^C P_V(k) \log P_V(k)$)
- The mutual information between U and V is as follows:
 - $MI(U, V) = \sum_{i=1}^R \sum_{k=1}^C P_{UV}(i, k) \log (P_{UV}(i, k)/(P_U(i) P_V(k)))$
 - where $P_{UV}(i, k) = |U_i \cap V_k|/m$
- Special case:
 - If p (any probability) = 0, then, we assume $p \log p = 0$
- $MI(U, V)$ is non-negative and upper bounded by $H(U)$ and $H(V)$

Example (I)

- Assume the following partitions (assignments):
 - $U = \{ U_1 = \{x^{(1)}, x^{(3)}, x^{(7)}\}, U_2 = \{x^{(4)}, x^{(5)}\}, U_3 = \{x^{(2)}, x^{(6)}, x^{(8)}\} \}$
 - $V = \{ V_1 = \{x^{(3)}, x^{(4)}, x^{(7)}\}, V_2 = \{x^{(2)}, x^{(5)}\}, V_3 = \{x^{(1)}\}, V_4 = \{x^{(6)}, x^{(8)}\} \}$
- $P_u(1)=3/8, P_u(2) = 2/8, P_u(3) = 3/8$
- $P_v(1)=3/8, P_v(2)=2/8, P_v(3)=1/8, P_v(4)=2/8$
- $H(U) = -(3/8 \log 3/8 + 2/8 \log 2/8 + 3/8 \log 3/8) = 1.08$
- $H(V) = -(3/8 \log 3/8 + 2/8 \log 2/8 + 1/8 \log 1/8 + 2/8 \log 2/8) = 1.32$

Example (II)

- Puv:

	V ₁	V ₂	V ₃	V ₄
U ₁	2/8	0/8	1/8	0/8
U ₂	1/8	1/8	0/8	0/8
U ₃	0/8	1/8	0/8	2/8

- $MI(U, V) = 2/8 \log 2/8/(3/8 * 3/8) + 0 + 1/8 \log 1/8/(3/8 * 1/8) + 0 + 1/8 \log 1/8/(2/8 * 3/8) + 1/8 \log 1/8/(2/8 * 2/8) + 0 + 0 + 0 + 1/8 \log 1/8/(3/8 * 1/8) + 0 + 2/8 \log 2/8/(3/8 * 2/8) = 0.76$

Example (III)

- Assume the following partitions (assignments):
 - $U = \{ U_1 = \{x^{(1)}, x^{(3)}, x^{(7)}\}, U_2 = \{x^{(4)}, x^{(5)}\}, U_3 = \{x^{(2)}, x^{(6)}, x^{(8)}\} \}$
 - $Y = \{ Y_1 = \{x^{(1)}\}, Y_2 = \{x^{(2)}\}, Y_3 = \{x^{(3)}, x^{(4)}\}, Y_4 = \{x^{(5)}, x^{(6)}\}, Y_5 = \{x^{(7)}\}, Y_6 = \{x^{(8)}\} \}$
- $P_U(1)=3/8, P_U(2) = 2/8, P_U(3) = 3/8$
- $P_Y(1)=1/8, P_Y(2)=1/8, P_Y(3)=2/8, P_Y(4)=2/8, P_Y(5)=1/8, P_Y(6)=1/8$
- $H(U) = -(3/8 \log 3/8 + 2/8 \log 2/8 + 3/8 \log 3/8) = 1.08$
- $H(Y) = -(1/8 \log 1/8 + 1/8 \log 1/8 + 2/8 \log 2/8 + 2/8 \log 2/8 + 1/8 \log 1/8 + 1/8 \log 1/8) = 1.73$

Example (IV)

- Puy:

	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6
U_1	1/8	0/8	1/8	0/8	1/8	0/8
U_2	0/8	0/8	1/8	1/8	0/8	0/8
U_3	0/8	1/8	0/8	1/8	0/8	1/8

- $MI(U, Y) = 1/8 \log 1/8/(3/8*1/8) + 0 + 1/8 \log 1/8/(3/8*2/8) + 0 + 1/8 \log 1/8/(3/8*1/8) + 0 + 0 + 0 + 1/8 \log 1/8/(2/8*2/8) + 1/8 \log 1/8/(2/8*2/8) + 0 + 0 + 0 + 1/8 \log 1/8/(3/8*1/8) + 0 + 1/8 \log 1/8/(3/8*2/8) + 0 + 1/8 \log 1/8/(3/8*1/8) = 0.74$

Distance based on MI

- Having partitions U, V and Y, we cannot compare their mutual information directly but using a metric
- $H(U) = 1.08$, $H(V) = 1.32$, $H(Y) = 1.73$, $MI(U, V) = 0.76$, $MI(U, Y) = 0.74$
- Distance: $D(U, V) = 1 - (MI(U, V) / \text{Max}(H(U), H(V)))$
- $D(U, V) = 0.42$, $D(U, Y) = 0.57$
- U and V are closer than U and Y

36

Adapted from: https://en.wikipedia.org/wiki/Mutual_information#Metric. The mutual information of U with respect to V and Y seem very similar. However, they are not if we consider the following distance.

Adjustment for chance (I)

Let S be a set of N data items, then a (partitional) clustering $\mathbf{U}$ on S is a way of partitioning S into non-overlap subsets $\{U_1, U_2, \dots, U_R\}$, where $\cup_{i=1}^R U_i = S$ and $U_i \cap U_j = \emptyset$ for $i \neq j$. The information on the overlap between two clusterings $\mathbf{U} = \{U_1, U_2, \dots, U_R\}$ and $\mathbf{V} = \{V_1, V_2, \dots, V_C\}$ can be summarized in form of a $R \times C$ contingency table $M = [n_{ij}]_{j=1, \dots, C}^{i=1, \dots, R}$ as illustrated in Table 1, where n_{ij} denotes the number of objects that are common to clusters U_i and V_j .

$\mathbf{U} \backslash \mathbf{V}$	V_1	V_2	$\dots$	V_C	Sums
U_1	n_{11}	n_{12}	$\dots$	n_{1C}	a_1
U_2	n_{21}	n_{22}	$\dots$	n_{2C}	a_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
U_R	n_{R1}	n_{R2}	$\dots$	n_{RC}	a_R
Sums	b_1	b_2	$\dots$	b_C	$\sum_{ij} n_{ij} = N$

Table 1: The Contingency Table, $n_{ij} = |U_i \cap V_j|$

From Vinh, Nguyen Xuan; Epps, Julien; Bailey, James (2010), "Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance" (PDF), The Journal of Machine Learning Research, 11 (oct): 2837–54

Adjustment for chance (II)

$$\mathbf{E}\{I(\mathbf{U}, \mathbf{V})\} = \sum_{i=1}^R \sum_{j=1}^C \sum_{n_{ij}=\max(a_i+b_j-N, 0)}^{\min(a_i, b_j)} \frac{n_{ij}}{N} \log\left(\frac{N \cdot n_{ij}}{a_i b_j}\right) \frac{a_i! b_j! (N - a_i)! (N - b_j)!}{N! n_{ij}! (a_i - n_{ij})! (b_j - n_{ij})! (N - a_i - b_j + n_{ij})!}$$

$$\text{AMI}(\mathbf{U}, \mathbf{V}) = \frac{I(\mathbf{U}, \mathbf{V}) - \mathbf{E}\{I(\mathbf{U}, \mathbf{V})\}}{\max\{H(\mathbf{U}), H(\mathbf{V})\} - \mathbf{E}\{I(\mathbf{U}, \mathbf{V})\}}$$

38

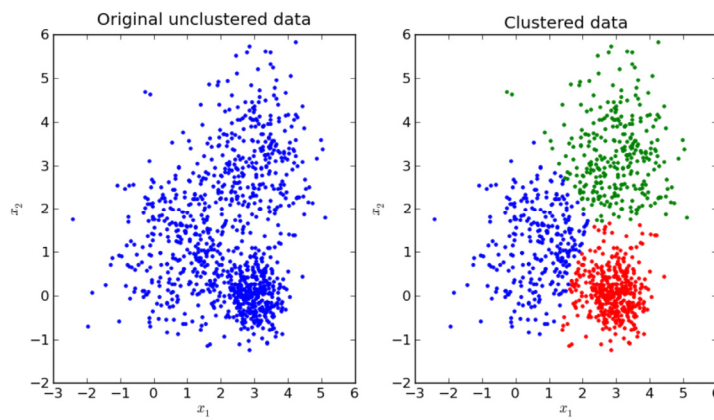
From Vinh, Nguyen Xuan; Epps, Julien; Bailey, James (2010), "Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance" (PDF), The Journal of Machine Learning Research, 11 (oct): 2837–54.

“To correct the measures for randomness it is necessary to specify a model according to which random partitions are generated. Such a common model is the “permutation model” (Lancaster, 1969, p. 214), in which clusterings are generated randomly subject to having a fixed number of clusters and points in each clusters.”

Roadmap

1. Introduction
2. The K-Means algorithm
3. Evaluating clusters
4. **Conclusions**

Grouping elements



40

We have studied a technique to cluster points together.

Fairness in clustering

- Group fairness for sensitive attributes (gender, ethnicity, religion...)
- Choosing a cluster with a high gender or ethnic skew for positive or negative actions, e.g., interview shortlisting or high scrutiny
- Could also lead to reinforcement of societal stereotypes: more black people go to jail

41

From Savitha Sam Abraham, Deepak P, Sowmya S. Sundaram: Fairness in Clustering with Multiple Sensitive Attributes. EDBT 2020: 287-298.

Thanks!

crr@cs.rit.edu

CARLOS RIVERO
ROCHESTER INSTITUTE OF TECHNOLOGY
DEPT. OF COMPUTER SCIENCE

R·I·T

Thank you very much for your attention.

Mutual information: exercises

- Assume the following partitions (assignments):
 - $U = \{ U_1 = \{x^{(1)}, x^{(3)}, x^{(7)}\}, U_2 = \{x^{(4)}, x^{(5)}\}, U_3 = \{x^{(2)}, x^{(6)}, x^{(8)}\} \}$
 - $V = \{ V_1 = \{x^{(3)}, x^{(4)}, x^{(7)}\}, V_2 = \{x^{(2)}, x^{(5)}\}, V_3 = \{x^{(1)}\}, V_4 = \{x^{(6)}, x^{(8)}\} \}$
- Assume the following partitions (assignments):
 - $U = \{ U_1 = \{x^{(1)}, x^{(3)}, x^{(7)}\}, U_2 = \{x^{(4)}, x^{(5)}\}, U_3 = \{x^{(2)}, x^{(6)}, x^{(8)}\} \}$
 - $Y = \{ Y_1 = \{x^{(1)}\}, Y_2 = \{x^{(2)}\}, Y_3 = \{x^{(3)}, x^{(4)}\}, Y_4 = \{x^{(5)}, x^{(6)}\}, Y_5 = \{x^{(7)}\}, Y_6 = \{x^{(8)}\} \}$
- Cheat sheet:
 - $P_U(i) = |U_i|/m$ (similarly $P_V(k) = |V_k|/m$)
 - $H(U) = -\sum_{i=1}^R P_U(i) \log P_U(i)$ (similarly $H(V) = -\sum_{k=1}^C P_V(k) \log P_V(k)$)
 - $MI(U, V) = \sum_{i=1}^R \sum_{k=1}^C P_{UV}(i, k) \log (P_{UV}(i, k)/(P_U(i) P_V(k)))$, $P_{UV}(i, k) = |U_i \cap V_k|/m$
 - If p (any probability) = 0, then, we assume $p \log p = 0$